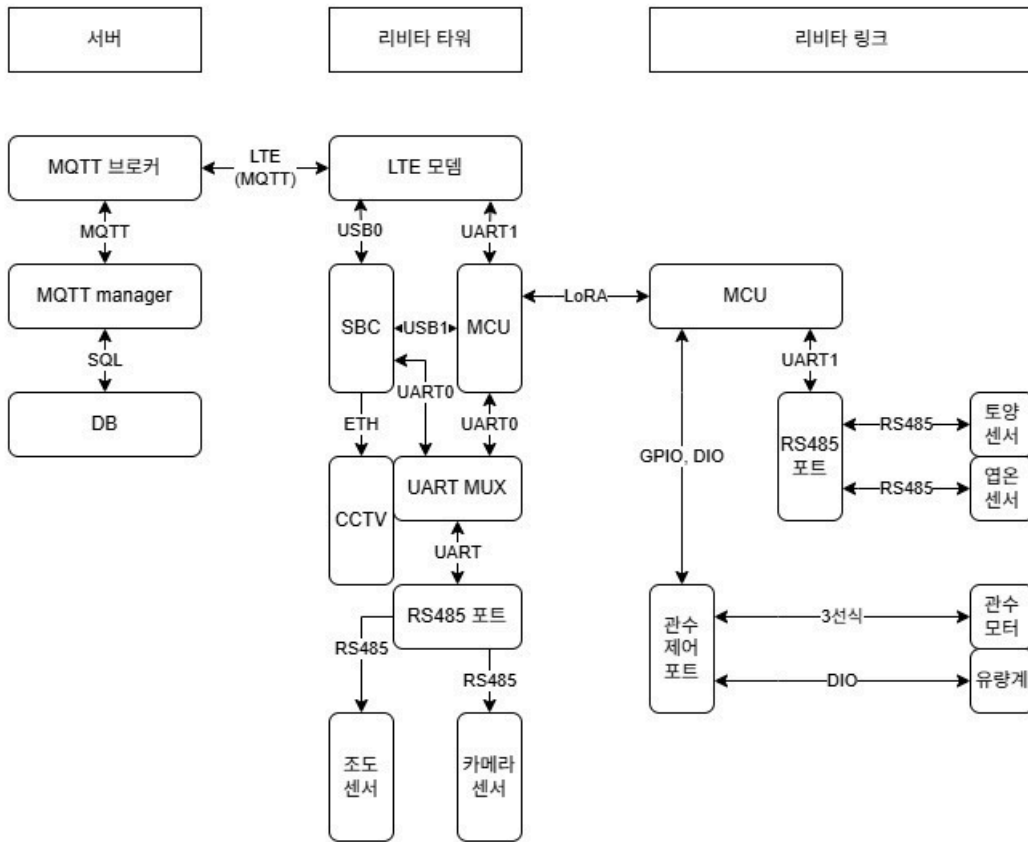


26-04-03 리비타 타워-링크 동작 시나리오 정리

구성



1. 서버 영역: 관제 및 데이터 저장

서버는 하드웨어의 상태를 추적하고 사용자의 명령을 현장으로 전달하는 중추 역할을 합니다.

- **MQTT 브로커 & 매니저**: 타워와 LTE를 통해 실시간 양방향 통신을 수행합니다. MQTT 프로토콜을 사용하여 저전력 환경에서도 안정적으로 메시지를 송수신합니다.
- **DB (SQL)**: 장비의 상태, 수집된 센서 데이터(raw data), 영농 활동 기록 등을 시계열로 저장하고 관리합니다.

2. 리비타 타워: 필지 중앙 게이트웨이

타워는 외부망(LTE)과 내부망(LoRA, RS485)을 잇는 핵심 장비로, 이중 프로세서(SBC + MCU) 구조를 가집니다.

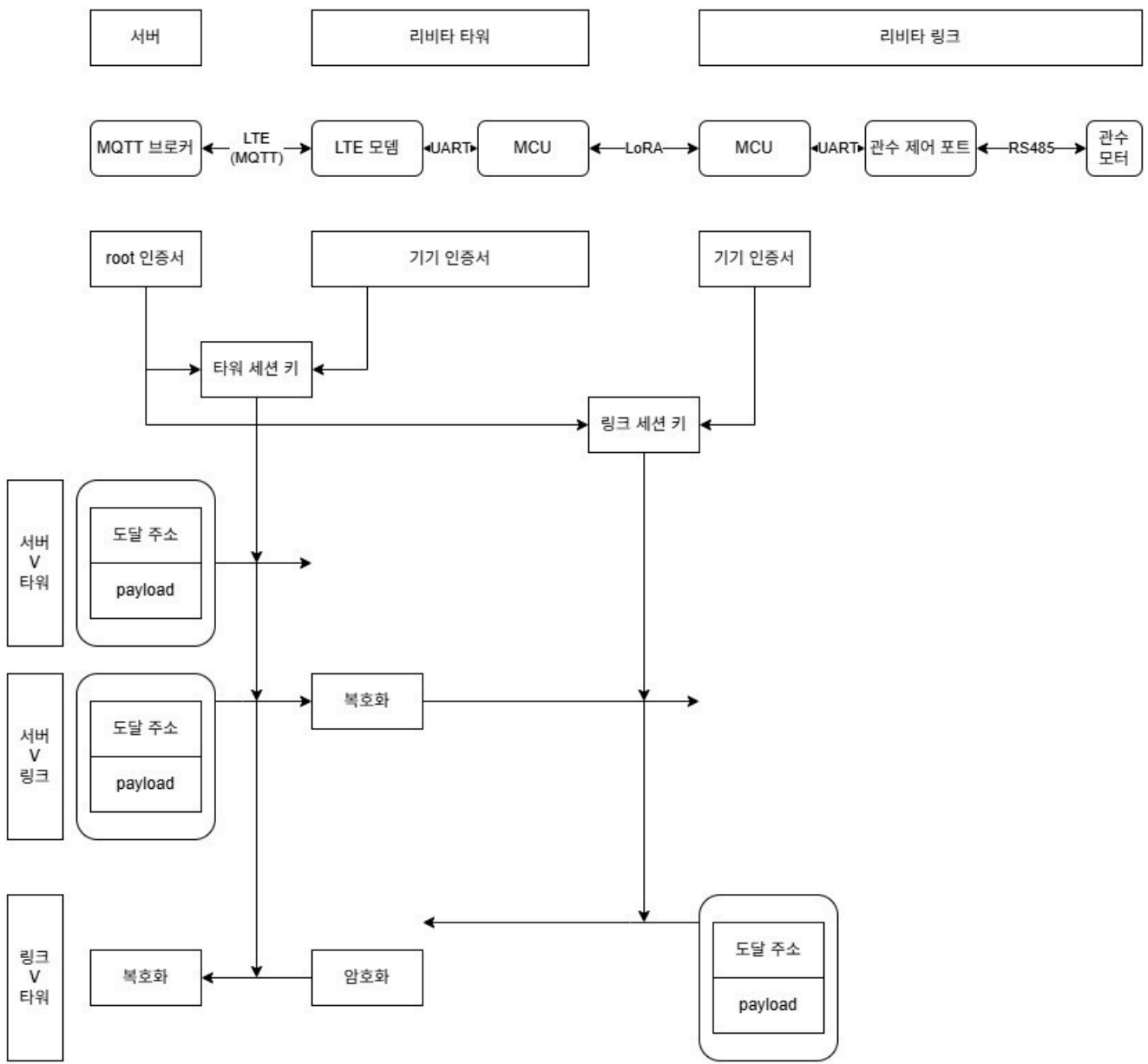
- **LTE 모뎀**: 서버와 통신하며, 내부적으로 고성능 처리가 필요한 **SBC(USB0)** 및 저전력 관리를 담당하는 MCU(UART1)와 각각 연결됩니다.
- **SBC (고성능 프로세서)**: PTZ 카메라를 통해 수집된 고해상도 영상을 가공하고 업로드하며, 분광 이미지를 분석합니다. CCTV와는 Ethernet(ETH)으로 연결되어 고속 데이터 전송을 보장합니다.
- **상시 구동 MCU (RAK4630)**: LoRA Mesh 망 관리 및 전원 스위칭을 담당합니다. 전력을 아끼기 위해 평소엔 MCU만 구동되다가, 필요 시에만 SBC와 센서에 전원을 공급합니다.
- **UART MUX & RS485 포트**: SBC와 MCU 중 우선순위에 따라 외부 센서를 제어할 수 있도록 경로를 제어합니다. 여기에 연결된 **조도 센서**는 일사량을 측정하고, **카메라/분광 센서**는 작물의 스트레스를 조기에 탐지합니다.

3. 리비타 링크: 현장 실행 노드

링크는 필지 곳곳에 배치되어 실제 농작업을 수행하며, 타워와 **LoRA Mesh**를 통해 무선으로 연결됩니다.

- **MCU (RAK4630):** 저전력 프로세서로 상시 구동되며, 타워로부터 수신된 명령에 따라 센서 측정이나 관수를 수행합니다.
- **RS485 포트 (UART1 연결):** 표준 프로토콜(Modbus RTU)을 통해 **토양 센서**(수분, pH, EC 측정) 및 **엽온/온습도 센서**와 통신합니다.
- **관수 제어 포트 (GPIO/DIO):** 관수 모터(3선식/릴레이) 및 유량계(DIO)와 연결되어 정밀한 관수 자동화를 구현합니다.

암호화 시스템 구성



1. 키 생성 단계: 신뢰의 분리

가장 눈에 띄는 변화는 '누가 누구와 열쇠를 만드느냐'입니다.

- **타워 세션 키 (Tower Session Key):** 서버 인증서 + 타워 기기 인증서를 조합해 만듭니다. 서버와 타워 사이의 LTE 통로를 지키는 열쇠입니다.
- **링크 세션 키 (Link Session Key):** 타워 기기 인증서 + 링크 기기 인증서를 조합해 만듭니다. 타워와 링크 사이의 LoRA 구간을 지키는 열쇠입니다.

2. 구간별 데이터 흐름 (4가지 시나리오)

① 서버 ↔ 타워 (관리 통신)

- 서버가 타워 자체를 제어할 때 사용합니다.
- **타워 세션 키**로만 암호화되어 주고받습니다.

② 서버 → 링크 (원격 제어)

- 서버가 모터를 돌리라고 명령을 내릴 때의 복잡한 과정입니다.
- **서버**: 명령어를 **타워 세션 키**로 잠가서 보냅니다.
- **타워**: 자기 열쇠로 금고를 엽니다. (**복호화**) → 이때 타워 내부 메모리에서 데이터는 잠깐 '**평문(Plaintext)**' 상태가 됩니다.
- **타워**: 내용을 확인한 뒤, 다시 **링크 세션 키**로 잠급니다. (**암호화**)
- **링크**: 타워가 보낸 암호를 풀어 명령을 실행합니다.

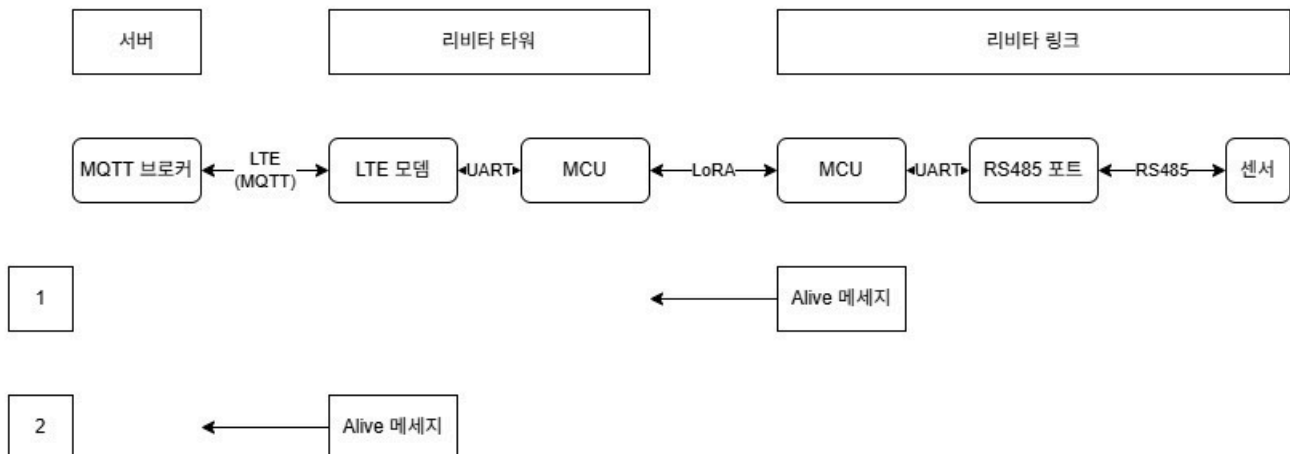
③ 링크 → 서버 (상태 보고)

- 모터가 일을 마치고 서버에 보고할 때입니다.
- **링크**: 데이터를 **링크 세션 키**로 잠가서 타워에 씁니다.
- **타워**: 링크 키로 데이터를 풉니다. (**복호화**) → 다시 '**평문**' 상태가 됩니다.
- **타워**: 서버가 알아들을 수 있게 **타워 세션 키**로 다시 잠가서 보냅니다. (**암호화**)
- **서버**: 최종 데이터를 확인합니다.

④ 링크 ↔ 타워 (로컬 자동화)

- 이번 구성에서 새로 명시된 강력한 기능입니다.
- 서버를 거치지 않고 **타워와 링크가 직접** 데이터를 주고받습니다.
- 서버 연결이 끊긴 비상 상황에서도 타워가 **링크 세션 키**를 가지고 있으므로, 현장을 안전하게 계속 제어할 수 있습니다.

리비타 링크 통신 유지 방식



1단계. [링크 → 타워] 저전력 기반 상태 전송 (LoRA)

- **10분 주기 체크**: 리비타 링크는 기본적으로 10분 단위로 센서 데이터를 측정하여 보고합니다. 만약 설정된 주기(10분) 이내에 사용자 명령이나 센서 값 전송 등 아무런 '토픽(Topic)'이 발생하지 않았을 경우, 링크 내 MCU가 스스로 장비의 생존 여부를 알리는 **Alive 메시지**를 생성합니다.
- **무선 송신**: 이 메시지는 저전력 장거리 통신 기술인 **LoRA(KR920)** 대역을 통해 리비타 타워로 무선 송신됩니다.

2단계. [타워 → 서버] 게이트웨이 중계 (LTE/MQTT)

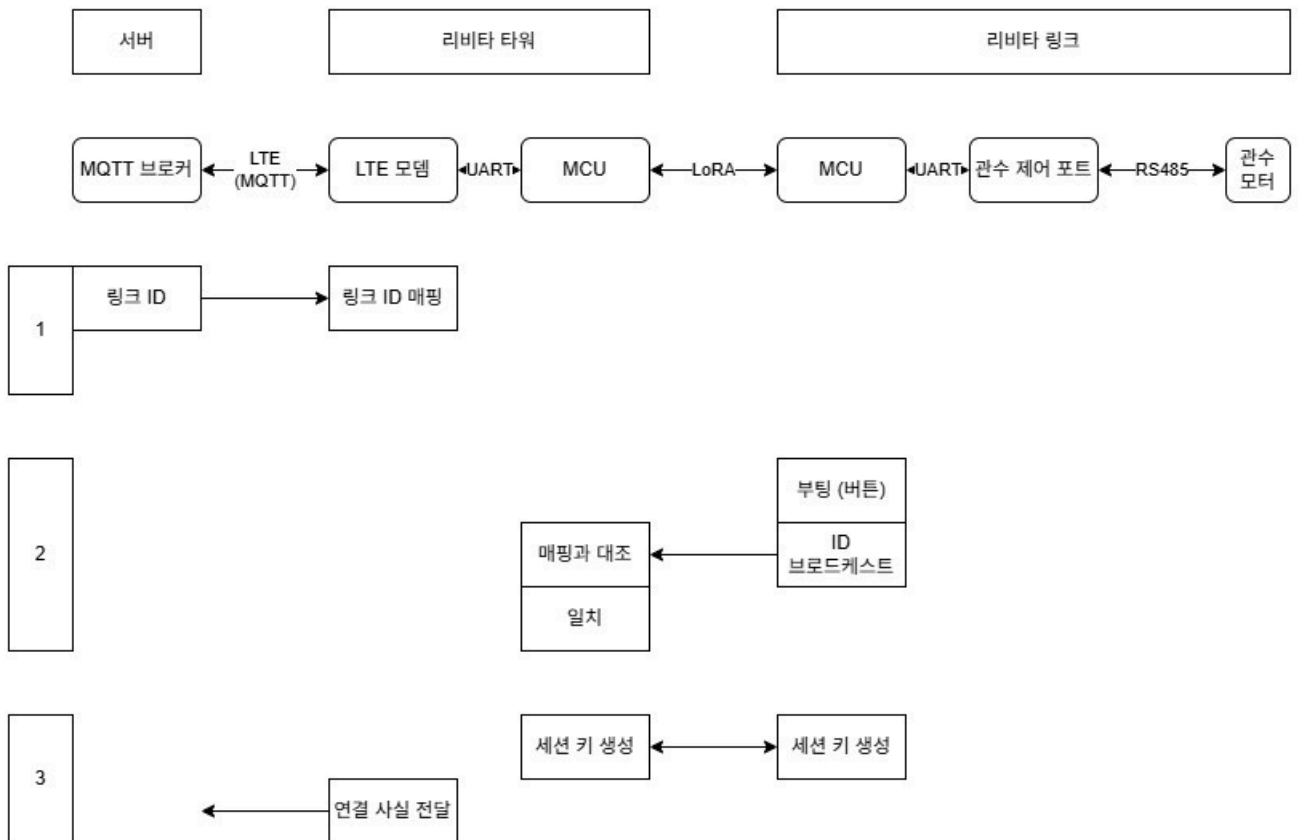
- **신호 수신 및 변환**: 리비타 타워는 필지 내의 중앙 관제 장비로서 링크가 보낸 LoRA 신호를 수신합니다. 타워 내 MCU는 이 신호를 해석한 뒤, LTE 모뎀을 통해 서버로 전달할 준비를 합니다.

- **MQTT 메시지 발행:** 타워는 **LTE(MQTT)** 프로토콜을 활용하여 서버의 브로커로 '링크 Alive' 신호를 중계합니다. 이는 백엔드 시스템에서 해당 링크가 현재 네트워크에 정상적으로 물려 있다는 것을 증명하는 데이터가 됩니다.

3단계. [서버 → DB] 상태 업데이트 및 모니터링

- **Last Seen 갱신:** 서버의 MQTT 매니저는 수신된 신호를 바탕으로 데이터베이스(DB) 내 해당 기기 레코드의 `last_seen` (최종 활성 시간) 타임스탬프를 현재 시간으로 업데이트합니다.
- **Reconcile(상태 복구):** 만약 DB 상태는 '관수 중(running)'인데 Alive 신호가 일정 시간 오지 않거나, 반대로 '대기(idle)' 상태인데 Alive 신호가 들어오는 경우, 서버는 이를 감지하여 장치 상태를 자동으로 동기화(Reconcile)합니다.

리비타 링크 통신 최초 구성



1단계: 기기 매핑 및 사전 등록 (서버 → 타워)

- **ID 전달:** 서버는 관리자 설정을 통해 특정 필지에 배정된 **링크 ID** 목록을 해당 리비타 타워로 전달합니다.
- **매핑 저장:** 리비타 타워는 이를 수신하여 내부 메모리에 **링크 ID 매핑** 데이터로 저장합니다. 이는 이후 현장에서 기기를 식별하는 '화이트리스트' 역할을 합니다.

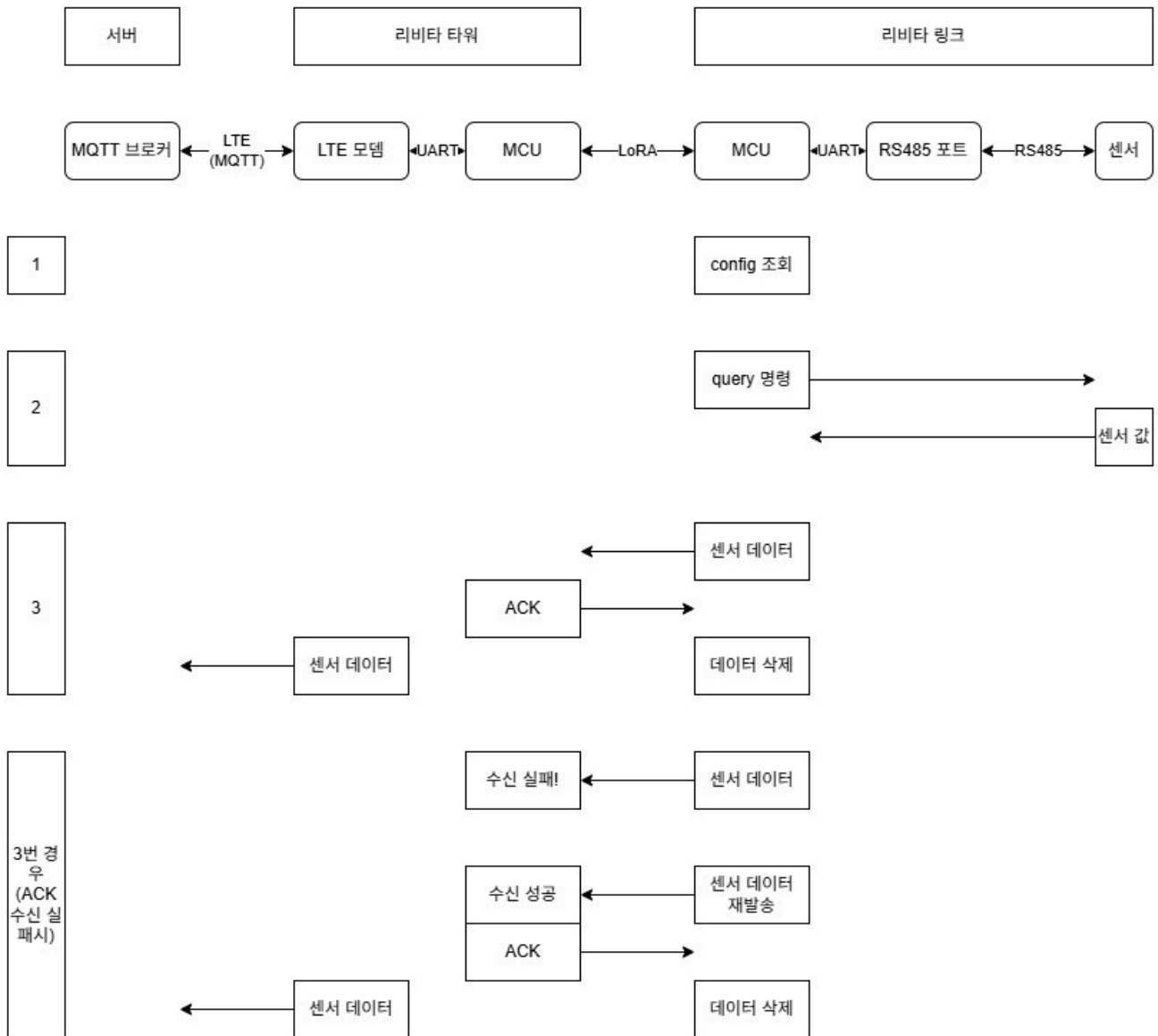
2단계: 부팅 및 ID 브로드캐스트 (링크 → 타워)

- **기기 가동:** 사용자가 리비타 링크의 전원 버튼을 눌러 **부팅**을 시작합니다.
- **조인 요청:** 링크는 자신의 고유 식별자를 담은 **ID 브로드캐스트** 패킷을 LoRA P2P 채널을 통해 주변으로 뿌립니다.
- **매핑 대조:** 리비타 타워는 이 신호를 수신한 즉시 1단계에서 저장한 데이터와 **대조**합니다.
- **[중요] 불일치 시 무시:** 대조 결과 ID가 일치할 때만 승인하며, **목록에 없는(불일치) 기기가 보낸 요청은 타워에서 즉각 무시하여 무단 접속을 차단**합니다.

3단계: 세션 확립 및 연결 보고 (타워 ↔ 링크 ↔ 서버)

- **세션 키 생성:** ID가 일치함이 확인되면, 타워와 링크는 향후 암호화 통신을 위해 각각 **세션 키를 생성**하여 보안 채널을 형성합니다.
- **서버 보고:** 타워는 해당 링크가 정상적으로 네트워크에 합류했다는 **연결 사실 전달** 메시지를 LTE(MQTT)를 통해 서버로 보냅니다.
- **활성화:** 서버는 이 신호를 받고 DB의 `last_seen` 을 갱신하여 시스템상에서 해당 링크를 '활성' 상태로 전환합니다.

리비타 링크 센서 값 읽기



1단계. MCU 기상 및 설정 로드 (Config 조회)

- **딥슬립 탈출:** 리비타 링크의 MCU(RAK4630)는 전력 소모를 최소화하기 위해 평소 딥슬립 상태를 유지하다가, 내부 RTC(실시간 시계) 타이머가 만료되는 순간 깨어납니다.
- **지능형 Config 참조:** MCU는 Flash 메모리에 저장된 **Config** 데이터를 읽어옵니다. 이 Config는 단순히 다음 수집까지의 '시간(수집 주기)'만 담고 있는 것이 아니라, **RS485 슬레이브 주소, 읽어올 레지스터 주소, 패킷 포맷**, 그리고 **센서에 직접 보낼 구체적인 쿼리(Query) 명령어** 그 자체를 포함하고 있습니다.

핵심 변경점: 이를 통해 서버에서 Config만 업데이트하면, 현장의 링크가 어떤 종류의 센서와 통신할지, 어떤 데이터를 요청할지를 원격에서 유연하게 변경할 수 있습니다.

2단계. RS485 읽기

- **RS485 명령 송신:** MCU는 전력 절감을 위해 상시 꺼져 있던 외부 기기 포트에 전원을 공급하고, **RS485 포트**를 통해 센서(토양 수분, pH, 온습도 등)에 쿼리 명령을 보냅니다.
- **데이터 수신:** 센서로부터 측정된 값을 디지털 데이터 형태로 수신합니다.

3단계. 무선 중계 및 확답(ACK) 프로세스

이 단계는 데이터가 소실되지 않도록 보장하는 핵심 핸드셰이크 과정입니다.

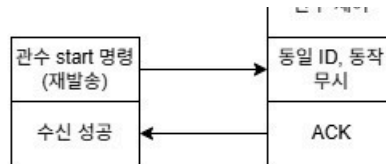
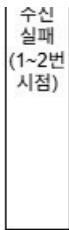
- **LoRA 전송:** 리비타 링크는 수집된 센서 데이터를 **LoRA 무선 통신**을 통해 필지 중앙의 리비타 타워로 송신합니다.
- **타워의 응답(ACK):** 리비타 타워는 데이터를 정상적으로 수신하면 링크에 수신 확인 신호(ACK)를 보냅니다.
- **데이터 확정 삭제:** 리비타 링크는 타워로부터 ACK를 받은 직후에만 내부 메모리에서 해당 데이터를 삭제하여, 전송 성공 전까지 데이터를 안전하게 보관합니다.
- **LTE/MQTT 중계:** 데이터를 수신한 리비타 타워는 **LTE(MQTT)** 통신을 활용하여 서버의 MQTT 브로커로 데이터를 즉시 발행합니다.
- **서버 저장:** 서버는 수신된 데이터를 DB에 시계열로 저장하고, 사용자의 웹/앱 대시보드에 반영합니다.

전송 실패 대응 로직 (Retry Mechanism)

이미지 하단의 3번 경우(ACK 수신 실패 시)는 통신 환경이 좋지 않을 때의 복구 시나리오를 설명합니다.

1. **수신 실패 감지:** 링크가 데이터를 보냈으나 타워로부터 ACK 신호를 받지 못하면 전송 실패로 간주합니다.
2. **데이터 재발송:** 리비타 링크는 삭제하지 않고 보관 중이던 센서 데이터를 **재발송**합니다.
3. **수신 성공 및 처리:** 타워가 재발송된 데이터를 정상 수신하면 링크에 ACK를 보내면, 그제야 링크는 데이터를 삭제하고 프로세스를 종료합니다.

리비타 링크 관수 제어



1. 정상 관수 시나리오 (단계 1 ~ 4)

단계 1: 관수 시작 명령 (Initiation)

- **서버**: 사용자가 앱을 통해 관수를 시작하면 서버는 MQTT 브로커를 통해 타워로 관수 시작 명령을 발행합니다. 이때 SQL DB의 상태는 `sent` 로 기록됩니다.
- **타워**: LTE 모뎀을 통해 명령을 수신한 뒤, 내부 MCU가 이를 LoRA 신호로 변환하여 해당 리비타 링크로 중계합니다.
- 관수 start 명령 payload
 - `link_hardware_id`: 타워 하나에 여러 개의 링크가 연결될 수 있으므로, 제어하고자 하는 특정 **리비타 링크의 ID**를 명시해야 합니다.
 - `command_id`: 각 명령을 식별하기 위한 고유한 **UUID**입니다. 서버는 이 ID를 통해 이후에 들어오는 ACK 나 `success` 응답이 어떤 명령에 대한 것인지 매칭하며, DB의 `pending_command_id` 에 저장합니다.
 - `execute_at`: 명령이 실행되어야 할 시점입니다. 일반적으로 즉시 실행을 위해 `0` 으로 설정됩니다.
 - `action`: 하드웨어가 수행해야 할 작업의 종류를 지정합니다.
 - `params` (세부 파라미터): 실제 제어에 필요한 핵심 정보들이 담깁니다.
 - `timer_minutes`: 관수를 지속할 시간(분)입니다. 이 값에 따라 **단계 4(타이머 도래)** 시점이 결정됩니다.

단계 2: 하드웨어 수신 확인 (Handshake)

- **링크**: 관수 시작 명령을 정상적으로 수신하면 타워로 ACK 신호를 보냅니다. 받은 `command ID`를 비휘발성 메모리에 저장해둡니다.
- **타워 & 서버**: 타워는 수신된 ACK를 서버로 포워딩합니다. 서버는 하드웨어가 명령을 받았음을 확인하고 SQL 상태를 `sent_ack` 로 업데이트합니다.
- ACK payload
 - `command_id`: 서버로부터 받은 관수 시작 명령의 **UUID와 동일한 값**을 반환해야 합니다. 서버는 이 ID를 대조하여 SQL DB의 `pending_command_id`와 매칭되는 행을 찾아 상태를 업데이트합니다.
 - `link_hardware_id`: 명령을 수신한 실제 **리비타 링크의 고유 ID**입니다. 타워가 여러 링크의 명령을 중계하므로, 어떤 노드에서 응답이 왔는지 명확히 식별하는 역할을 합니다.
 - `status`: 현재 하드웨어의 처리 단계를 나타내며, 단계 2에서는 반드시 **"ack"** 값을 담아야 합니다.
 - `finished_at`: 펌웨어가 해당 명령을 수신하고 확인(ACK)을 생성한 시점의 **ISO 8601 타임스탬프**입니다 (UTC 기준, 'Z' 접미사 포함).

단계 3: 관수 실행 (Execution)

- **링크**: 실제 관수 제어 포트를 통해 모터를 가동하여 밸브를 엽니다(**open**). 제어에 성공하면 `success` 신호를 타워로 보냅니다.
- **서버**: 성공 신호를 최종 확인하면 실제 관수가 진행 중임을 뜻하는 `running` 상태로 변경하며, 이때 시작 시간(`startedAt`)이 타임스탬프로 기록됩니다.
- success payload
 - `command_id`: 앞선 start 및 ack 단계와 동일한 **UUID**를 유지합니다. 서버는 이 ID를 통해 `sent_ack` 상태로 대기 중이던 특정 명령이 성공했음을 인지합니다.

- link_hardware_id: 실제 관수 제어를 수행한 **리비타 링크의 ID**입니다.
- status: 하드웨어 제어가 물리적으로 성공했음을 뜻하는 "**success**" 값을 담습니다.
- finished_at: 단순히 신호를 보낸 시간이 아니라, **실제로 밸브/모터가 열린 시점**의 타임스탬프입니다.

단계 4: 타이머 종료 (Completion)

- **링크**: 설정된 관수 시간(타이머)이 종료되면 스스로 밸브를 닫고(**close**) 종료를 알리는 end 신호를 보냅니다.
- **서버**: 종료 신호를 수신하면 관수 프로세스가 정상적으로 마무리되었음을 확인하고 SQL 상태를 ending_ack 로 기록합니다. 이후 해당 행은 삭제되거나 이력(Log)으로 남습니다.
- end payload
 - command_id: 앞선 단계와 동일한 **UUID**를 유지합니다. 서버는 이 ID를 통해 진행 중이던 관수 명령이 정상 종료되었음을 인지합니다.
 - status: 관수가 정상적으로 종료되었음을 뜻하는 "**end**" 값을 담습니다.
 - finished_at: 실제 밸브가 **Close** 된 정확한 시점의 UTC 타임스탬프입니다. 서버는 이 값을 바탕으로 실제 관수 시간을 계산해 영농일지에 기록합니다.

2. 예외 시나리오 (동작 실패 및 수동 종료)

동작 실패 (3번 시작 시점)

- **링크**: 관수를 시도했으나 배터리 부족 등으로 모터 제어에 실패할 경우 failed 신호를 보냅니다.
- **서버**: 실패 리포트를 수신하면 SQL 상태를 failed 로 기록하고 사용자에게 오류 알림을 보냅니다.
- failed payload
 - command_id: 앞선 start 및 ack 단계와 동일한 **UUID**를 유지합니다. 서버는 이 ID를 통해 sent_ack 상태로 대기 중이던 특정 명령이 **실패**했음을 인지합니다.
 - status: 제어 명령 수행에 실패했음을 뜻하는 "**failed**" 값을 담습니다.
 - reason: 실패한 구체적인 원인을 전달합니다 (예: battery_low, motor_error, firmware_error).
 - finished_at: 실패가 확정된 시점의 타임스탬프입니다.

동작 중 종료 (3~4번 사이 시점)

- **서버**: 사용자가 앱에서 수동으로 중지를 누르면 서버는 중지 MQTT 명령을 발행하고 SQL 상태를 ending 으로 변경합니다.
- **링크**: 중지 명령 수신 시 즉시 ACK 를 보내 서버 상태를 ending_ack 로 만들고, 이어서 밸브를 닫은(**close**) 뒤 최종 end 신호를 보냅니다.
- **최종**: 서버는 모든 정지 확인이 완료되면 SQL 상태를 ended (또는 최종 완료 처리)로 확정합니다.
- end payload (수동 중지 후)
 - command_id: 앞선 단계와 동일한 **UUID**를 유지합니다. 서버는 이 ID를 통해 수동 중지에 따른 종료가 완료되었음을 인지합니다.
 - status: 관수가 정상적으로 종료되었음을 뜻하는 "**end**" 값을 담습니다.
 - finished_at: 실제 밸브가 **Close** 된 정확한 시점의 UTC 타임스탬프입니다. 서버는 이 값을 바탕으로 실제 관수 시간을 계산해 영농일지에 기록합니다.

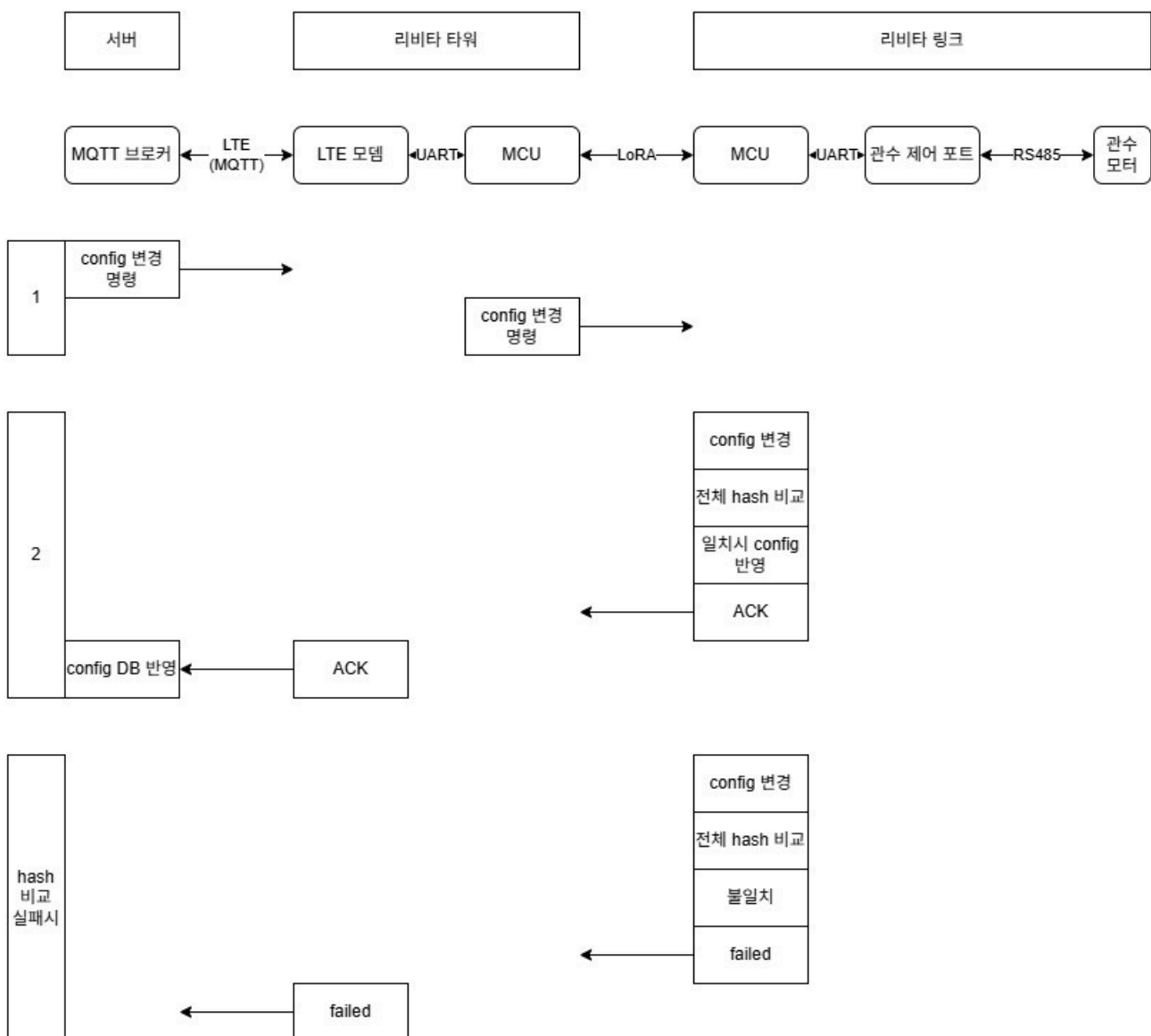
ACK 수신 실패 및 재발송 처리 시나리오

이 과정은 서버(타워)가 보낸 명령은 링크에 도착했지만, 링크가 보낸 응답(ACK)을 타워가 받지 못했을 때의 복구 흐름입니다.

- **리비타 타워**가 '관수 start 명령'을 보냅니다.

- 리비타 링크는 이를 정상 수신하여 **MCU 메모리에 해당 명령의 ID(UUID)를 등록**합니다. 이는 나중에 중복 명령을 판별하는 기준이 됩니다.
 - 링크는 명령을 받았다는 신호인 **ACK** 를 보냅니다. 하지만 통신 장애로 인해 타워 쪽에서 '수신 실패!'가 발생합니다. 이 시점에서 타워(서버)는 여전히 SQL 상태를 (sent) 로 유지하며 링크가 명령을 받지 못했다고 판단합니다.
 - 일정 시간 동안 ACK를 받지 못한 타워는 명령을 다시 보냅니다(**관수 start 명령 재발송**).
 - 재발송된 명령을 받은 링크는 메모리에 이미 저장된 ID와 대조합니다.
 - **동일 ID임을 확인하면**, 이미 첫 번째 명령에 의해 관수 제어(Open) 로직이 수행 중이거나 대기 중이므로 **추가 동작을 무시**합니다.
- 기획 의도:** 이 처리가 없으면 밸브가 중복으로 열리거나, 관수 타이머가 초기화되는 등 오작동이 발생할 수 있습니다.
- 링크는 동작은 무시하되, 타워가 상태를 업데이트할 수 있도록 다시 한번 **ACK** 를 보냅니다.
 - 타워가 이 ACK를 받으면 비로소 '수신 성공'으로 간주하고 서버 SQL 상태를 sent_ack 로 전이시킵니다.

리비타 링크 config 변경



단계 1: Config 변경 명령 하달 (Push)

- **서버:** 관리자가 설정을 변경하면 서버는 MQTT를 통해 'config 변경 명령'을 발행합니다. 이때 페이로드에는 변경될 설정 내용과 적용 후 기대되는 **전체 Hash 값**이 포함됩니다.
- **타워:** LTE로 수신한 명령을 LoRA망을 통해 해당 리비타 링크로 **중계(Forwarding)**합니다.

- config 변경 payload
 - expected_hash: 서버가 계산한 "이 설정이 적용되었을 때의 전체 Config Hash"입니다. 링크는 이 값을 기준으로 동기화 여부를 판단합니다.
 - config_data: 변경될 config의 주소, 데이터를 포함합니다.

단계 2: Config 반영 및 Hash 검증 (성공 시)

- **링크:** MCU는 수신한 설정을 Flash 메모리에 임시 기록하고, 현재 전체 설정에 대한 **Hash를 계산**합니다.
- **비교 및 반영:** 계산된 Hash와 서버가 보낸 **기대 Hash 값이 일치**하면, 링크는 설정을 확정(reflect)하고 ACK 신호를 보냅니다.
- **서버:** 타워를 통해 ACK를 수신하면 SQL DB의 장치 설정 상태를 '동기화 완료'로 업데이트합니다.

Hash 비교 실패 시 (예외 상황)

- **링크:** 계산된 Hash가 서버의 기대값과 **불일치**할 경우, 데이터가 전송 중 손상되었거나 누락된 것으로 간주합니다.
- **실패 보고:** 설정을 반영하지 않고 failed 신호를 보내며, 서버는 이를 통해 재전송 로직을 수행하거나 관리자에게 알립니다.